

Guía del Instructor

Módulo II · Sesión de cierre — ETL con SQL en Databricks y automatización con Jobs

■ Objetivo de la sesión

Que el estudiante construya, en SQL puro, un flujo ETL completo siguiendo la arquitectura medallion (bronce → plata → oro), y que comprenda qué es un pipeline y cómo automatizarlo con un Job de Databricks. Al cierre, el estudiante debe poder replicar el proceso de forma autónoma con un dataset nuevo.

Pregunta guía del proyecto transversal

¿Cómo se comporta el valor de las declaraciones de importación y su relación con el recaudo, por subpartida arancelaria y país de origen? — esta sesión aporta la base técnica (ETL + automatización) que sostiene todo el análisis posterior del proyecto.

Materiales de esta sesión

- **Dataset demo (instructor):** declaraciones_tributarias_500.parquet — 500 filas, con nulos, duplicados, inconsistencias de texto y valores negativos inválidos, diseñados intencionalmente para la práctica de limpieza.
- **Dataset del reto (estudiantes):** declaraciones_importacion_500.parquet — mismo estilo de "suciedad", tema de importaciones (conecta con el hilo transversal).
- **Notebook demo:** 01_ETL_Declaraciones_Tributarias.py (formato Databricks importable) / .sql (versión limpia sin comandos mágicos).
- **Diapositivas:** 02_ETL_y_Jobs_Modulo_II.pptx.
- **Datos sintéticos únicamente** — en ningún caso se usa información real de contribuyentes de la DIAN.

Guion de la demo — ETL en SQL

Sigan el notebook 01_ETL_Declaraciones_Tributarias en orden. La tabla siguiente resume qué mostrar y qué explicar en cada paso — la idea es que cada transformación se conecte explícitamente con el problema de calidad que resuelve (visto en el perfilamiento).

Paso	Qué corre	Qué explicar en voz alta
1. EXTRACT	CREATE TABLE bronze_... USING PARQUET	“Bronce es la copia fiel del origen, no transformamos nada todavía.”
2. EXPLORE	Conteos, nulos, distinct, negativos	“Nunca limpiamos a ciegas: primero medimos qué tan sucio está el dato.”
3. TRANSFORM	TRIM, INITCAP, COALESCE, ABS, ROW_NUMBER	Conectar cada función con el hallazgo del paso 2, uno a uno.

4. VALIDATE	Comparación de conteos bronce vs. plata	“No se confía a ciegas en una transformación: se valida.”
5. LOAD	CREATE TABLE gold_... agregada	“La capa oro responde directamente la pregunta de negocio.”

■ Tip de dictado

Antes de correr el bloque TRANSFORM, pregunten al grupo: "¿qué le harían ustedes a cada columna sucia que acabamos de ver?" — casi siempre alguien propone TRIM o quitar duplicados antes de que ustedes lo muestren. Eso ancla el concepto mucho mejor que solo mostrar el código.

Guion de la demo — Creación del Job (en vivo)

- 1 En el panel izquierdo: **Workflows** → **Jobs** → **Create Job**.
- 2 Nombrar el Job, ej: *etl_declaraciones_tributarias*.
- 3 Agregar una Task de tipo **Notebook**, apuntando a 01_ETL_Declaraciones_Tributarias.
- 4 Seleccionar o crear el cluster que ejecutará la Task (mencionar que en producción suele ser un job cluster efímero, no uno interactivo).
- 5 Dejar el Trigger en **Manual** para la demo — mencionar "Scheduled" (cron) como el siguiente nivel, sin necesidad de configurarlo en vivo.
- 6 Click en **Run now** y observar el estado en tiempo real (Pending → Running → Succeeded).
- 7 Abrir el **historial de Runs** y mostrar los logs de esa ejecución — este es el "aha" de la sesión: el Job queda documentado solo.

■ Si algo falla en vivo

El error más común es de permisos o de ruta del volumen ("path not found"). Tengan lista, como respaldo, una captura de pantalla de un Run exitoso previamente ejecutado, para no perder el hilo de la clase si el cluster tarda en levantar.

Solución del reto — ETL de declaraciones de importación

Esta es la solución de referencia para calificar el reto de los estudiantes. La lógica es idéntica a la demo, aplicada al dataset *declaraciones_importacion_500.parquet*.

```

CREATE OR REPLACE TABLE bronze_declaraciones_importacion
USING PARQUET
OPTIONS (path '/Volumes/.../declaraciones_importacion_500.parquet');

CREATE OR REPLACE TABLE silver_declaraciones_importacion AS
WITH dedup AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY id_declaracion ORDER BY fecha_declaracion DESC) AS rn
  FROM bronze_declaraciones_importacion
)
SELECT
  id_declaracion, id_importador,
  INITCAP(TRIM(tipo_importador)) AS tipo_importador,
  INITCAP(TRIM(departamento)) AS departamento,
  INITCAP(TRIM(ciudad)) AS ciudad,
  subpartida_arancelaria, descripcion_producto,
  INITCAP(TRIM(pais_origen)) AS pais_origen,
  anio_declaracion, valor_fob,
  ABS(fletes) AS fletes,
  COALESCE(seguros, 0) AS seguros,
  valor_cif, ABS(arancel) AS arancel,
  iva, valor_total_pagar, estado_declaracion, fecha_declaracion
FROM dedup WHERE rn = 1;

CREATE OR REPLACE TABLE gold_importaciones_por_subpartida AS
SELECT subpartida_arancelaria, pais_origen, anio_declaracion,
  COUNT(*) AS num_declaraciones,
  SUM(valor_fob) AS total_valor_fob,
  SUM(valor_total_pagar) AS total_pagado
FROM silver_declaraciones_importacion
GROUP BY subpartida_arancelaria, pais_origen, anio_declaracion
ORDER BY total_valor_fob DESC;

```

Rúbrica sugerida (20 pts)

Criterio	Puntos
Tabla bronce creada correctamente desde el archivo parquet	3
Perfilamiento documentado (nulos, duplicados, inconsistencias)	4
Tabla plata: normalización de texto + manejo correcto de nulos/negativos	6
Tabla oro responde la pregunta de negocio planteada	4
Job creado y ejecutado exitosamente (captura del Run)	3